

Aufgabenblatt 8

Aufgabe 8.1 i)

MatMul

```
OpenCL - elapsed time: 529 ms
Scalar - elapsed time: 44412 ms
The matrices are the same within
tolerance.
```

FLOPs:

Jede Ausgabezelle erfordert N Multiplikationen mit $N - 1$ Additionen. Da wir $N * N$ Ausgabezellen haben gilt somit:

$$N * N * (N + N - 1) \approx 2N^3 \text{ Flops.}$$

In unserem Fall $2 * 2048^3 \approx 1.7 * 10^{10}$ Flops

Gesamtanzahl Elemente bei Datenübertragung zwischen RAM und VRAM:

$$3 * N^2 + 1 = O(N^2)$$

Die OpenCL-Version ist über 80-mal schneller als die skalare Implementierung

MatAdd

```
OpenCL - elapsed time: 270 ms
Scalar - elapsed time: 2 ms
The matrices are the same within
tolerance.
```

FLOPs:

$N * N$ Additionen, somit N^2 Flops.

In unserem Fall $2048^2 \approx 4 * 10^6$ Flops

Gesamtanzahl Elemente bei Datenübertragung zwischen RAM und VRAM:

$$3 * N^2 + 1 = O(N^2)$$

Die Datenübertragung braucht schon mehr Schritte als die skalare Berechnung. D.h. die Parallelisierung würde keine Vorteile bringen.

MatMul

Arithmetische Intensität (AI)

$$AI = \frac{\text{Anzahl FLOps}}{\text{Anzahl der übertragenen Bytes}}$$
$$= \frac{2 * N^3}{3 * 4 * N^2} = \frac{N}{6} \left[\frac{\text{FLOPs}}{\text{Byte}} \right]$$

→ gut für GPU

MatAdd

Arithmetische Intensität (AI)

$$AI = \frac{\text{Anzahl FLOps}}{\text{Anzahl der übertragenen Bytes}}$$
$$= \frac{N^2}{3 * 4 * N^2} = \frac{1}{12} \left[\frac{\text{FLOPs}}{\text{Byte}} \right]$$

→ schlecht für GPU

Bei der Matrixaddition ist der Rechenaufwand im Vergleich zum Datentransfer sehr gering. Es müssen drei Matrizen der Größe $N \times N$ zwischen RAM und VRAM übertragen werden (zwei Eingabematrizen und eine Ausgabematrix), was $3 * N^2$ Speichertransfers erfordert. Demgegenüber stehen lediglich N^2 skalare Operationen (eine Addition pro Matrixelement). Der Overhead für den Datentransfer sowie die Kernel-Verwaltung überwiegen hier den tatsächlichen Rechenaufwand, sodass sich die Nutzung von OpenCL nicht lohnt.

Anders bei der Matrixmultiplikation: Hier beträgt der Rechenaufwand etwa $2 * N^3$ FLOPs, während der Datentransfer ebenfalls $3 * N^2$ Elemente betrifft. Da die Anzahl der Rechenoperationen im Vergleich zum Datentransfer deutlich höher ist (arithmetische Intensität $\approx \frac{2 * N^3}{3 * N^2} = \frac{2}{3} N$), ist die Auslastung der GPU wesentlich effizienter. Die Parallelisierung über viele Threads amortisiert den Übertragungsaufwand, weshalb OpenCL hier deutlich bessere Resultate liefert.

Aufgabe 8.1 ii)

Wie groß ist unser VRAM von unserem GPU?:

```
paris:Code> clinfo | grep "Global memory size"
Global memory size          3893690368 (3.626GiB)
```

=> 3893690368 Bytes

Da wir $3 * N^2$ viele Floats (4 Bytes pro Float) senden, muss gelten:

$$3 * N^2 * 4 \leq 3893690368 \Leftrightarrow N \leq \sqrt{3893690368/12} \approx 18013$$

$$2^{15} = 32768 > 18013$$

$$2^{14} = 16384 \leq 18013$$

$$\Rightarrow \text{maximum } N := 2^{14} = 16384$$

N = 16:

```
OpenCL - elapsed time: 260 ms
Scalar - elapsed time: 0 ms
OpenMP + SIMD - elapsed time: 0 ms
OpenCL and scalar matrices are the same within tolerance.
OpenMP and scalar matrices are the same within tolerance.
```

N = 32:

```
OpenCL - elapsed time: 269 ms
Scalar - elapsed time: 0 ms
OpenMP + SIMD - elapsed time: 0 ms
OpenCL and scalar matrices are the same within tolerance.
OpenMP and scalar matrices are the same within tolerance.
```

N = 64:

```
OpenCL - elapsed time: 283 ms
Scalar - elapsed time: 0 ms
OpenMP + SIMD - elapsed time: 0 ms
```

```
OpenCL and scalar matrices are the same within tolerance.  
OpenMP and scalar matrices are the same within tolerance.
```

```
N = 128:
```

```
OpenCL - elapsed time: 257 ms  
Scalar - elapsed time: 1 ms  
OpenMP + SIMD - elapsed time: 0 ms  
OpenCL and scalar matrices are the same within tolerance.  
OpenMP and scalar matrices are the same within tolerance.
```

```
N = 256:
```

```
OpenCL - elapsed time: 272 ms  
Scalar - elapsed time: 16 ms  
OpenMP + SIMD - elapsed time: 2 ms  
OpenCL and scalar matrices are the same within tolerance.  
OpenMP and scalar matrices are the same within tolerance.
```

```
N = 512:
```

```
OpenCL - elapsed time: 273 ms  
Scalar - elapsed time: 201 ms  
OpenMP + SIMD - elapsed time: 6 ms  
OpenCL and scalar matrices are the same within tolerance.  
OpenMP and scalar matrices are the same within tolerance.
```

```
N = 1024:
```

```
OpenCL - elapsed time: 331 ms  
Scalar - elapsed time: 1696 ms  
OpenMP + SIMD - elapsed time: 47 ms  
OpenCL and scalar matrices are the same within tolerance.  
OpenMP and scalar matrices are the same within tolerance.
```

```
N = 2048:

OpenCL - elapsed time: 720 ms
Scalar - elapsed time: 47352 ms
OpenMP + SIMD - elapsed time: 468 ms
OpenCL and scalar matrices are the same within tolerance.
OpenMP and scalar matrices are the same within tolerance.
```

```
N = 4096:

OpenCL - elapsed time: 3870 ms
Scalar - elapsed time: 473471 ms
OpenMP + SIMD - elapsed time: 4221 ms
OpenCL and scalar matrices are the same within tolerance.
OpenMP and scalar matrices are the same within tolerance.
```

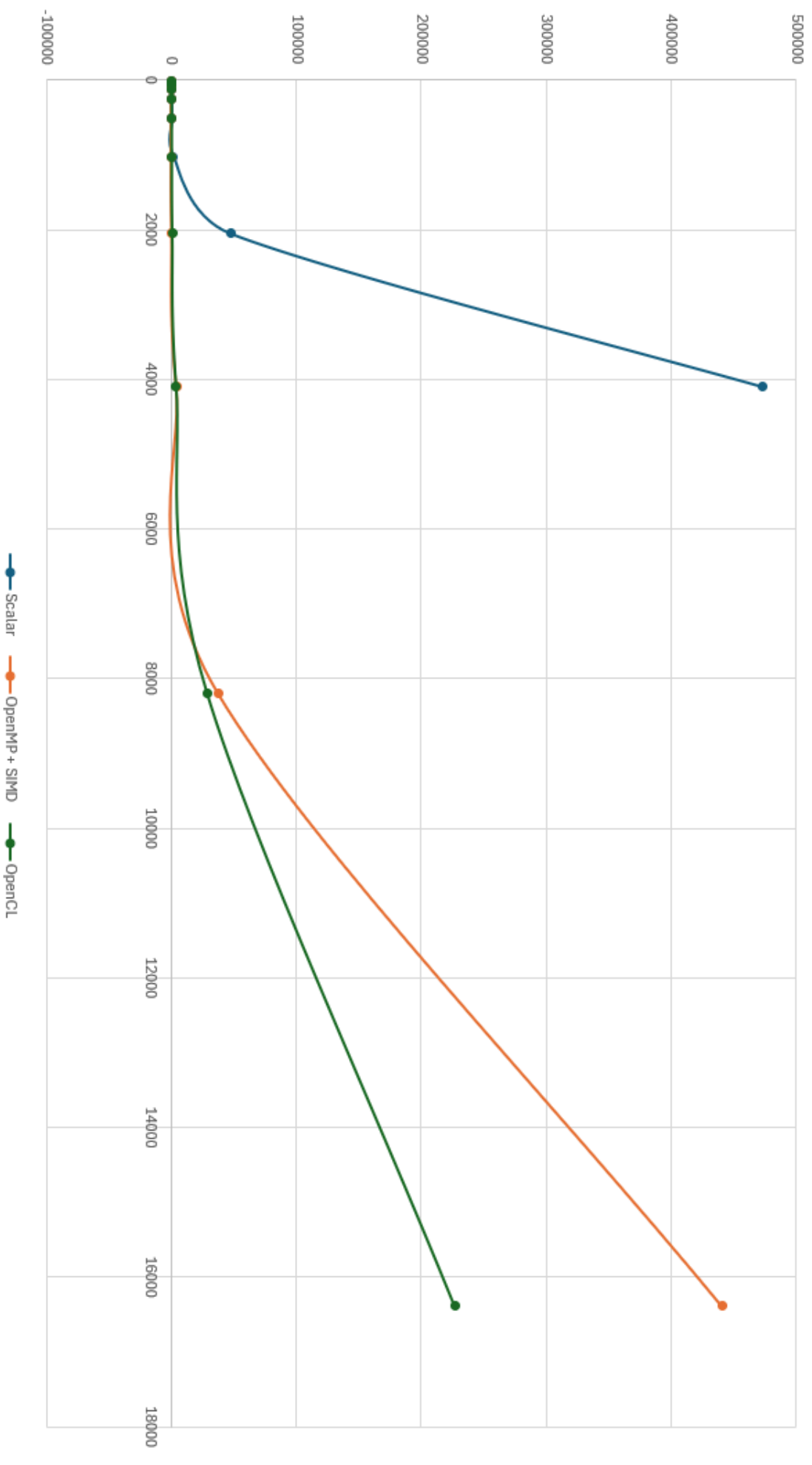
Die skalare Version wird ab $N = 8192$ unpraktisch langsam, da die Laufzeit kubisch ansteigt. Um Zeit und Systemressourcen zu sparen, haben wir für größere N keine skalaren Laufzeiten mehr gemessen:

```
N = 8192:

OpenCL - elapsed time: 28701 ms
N too large for scalar version.
OpenMP + SIMD - elapsed time: 37654 ms
OpenMP and OpenCL matrices are the same within tolerance.
```

```
N = 16384:

OpenCL - elapsed time: 227025 ms
N too large for scalar version.
OpenMP + SIMD - elapsed time: 441025 ms
OpenMP and OpenCL matrices are the same within tolerance.
```



$N \leq 128$:

Alle Zeiten von Scalar und OpenMP (+ SIMD) sind sehr niedrig (teilweise 0 ms, da unter der Messgenauigkeit). OpenCL ist hier nicht schneller als die CPU: Der Overhead für das Übertragen der Daten auf den VRAM der GPU und das Starten des Kernels ist bei kleinen Matrizen größer als der eigentliche Rechenaufwand. Deshalb ist OpenCL hier sogar langsamer als OpenMP und Scalar.

$256 \leq N \leq 1024$:

OpenMP ist deutlich schneller als Scalar: Die CPU kann Multithreading und SIMD ausnutzen. OpenCL bleibt etwa gleich schnell. Der Overhead ist immer noch spürbar, aber die GPU kann ihre Vorteile noch nicht voll ausnutzen. Scalar wird langsam, denn die Laufzeit steigt stark an, da keine Parallelisierung genutzt wird.

$N \geq 2048$:

Scalar explodiert, denn die Laufzeit wächst exponentiell und jede Operation wird einzeln auf der CPU ausgeführt. OpenMP bleibt deutlich schneller als Scalar, aber die Laufzeit steigt ebenfalls stark an, da der Speicherbedarf zunimmt. OpenCL wird jetzt schneller als OpenMP. Bei sehr großen Matrizen kann die GPU ihre massive Parallelität ausnutzen und der Overhead fällt im Vergleich zur Rechenzeit kaum noch ins Gewicht.

$N \geq 8192$:

OpenMP und OpenCL werden beide sehr langsam, aber OpenCL ist jetzt klar im Vorteil, weil die GPU für große Datenmengen optimiert ist. OpenMP ist limitiert durch die Anzahl der CPU-Kerne. OpenCL profitiert von der hohen Parallelität der GPU, aber auch hier steigen die Zeiten stark an, da der Speicherbedarf riesig ist und die Datenübertragung zur GPU viel Zeit kostet.